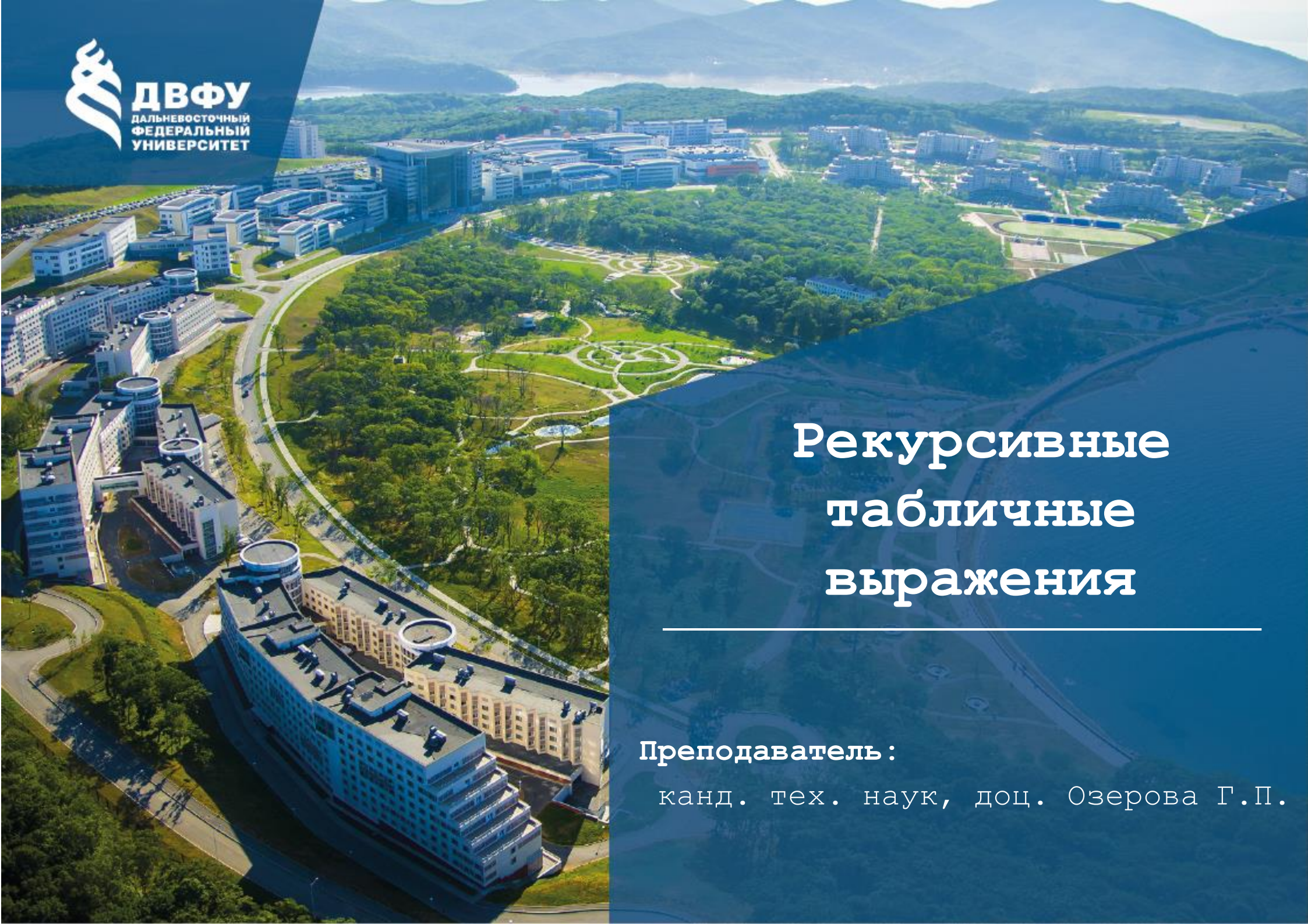


Рекурсивные табличные выражения

Преподаватель:

канд. тех. наук, доц. Озерова Г.П.

Рекурсия

С помощью табличных выражений можно написать рекурсивный запрос, т.е. запрос, который, написанный один раз, будет повторяться многократно пока истинно некоторое условие.

Рекурсия

Рекурсивное табличное выражение имеет следующий вид:

```
WITH RECURSIVE имя(список столбцов)
AS(




)
```

Рекурсия

Рекурсивное табличное выражение имеет следующий вид:

```
WITH RECURSIVE имя(список столбцов)
AS(
 SELECT... -- начальное значение



)
```

Рекурсия

Рекурсивное табличное выражение имеет следующий вид:

```
WITH RECURSIVE имя(список столбцов)
AS(
 SELECT... -- начальное значение
UNION ALL -- рекурсивная часть


)
```

Рекурсия

Рекурсивное табличное выражение имеет следующий вид:

```
WITH RECURSIVE имя(список столбцов)
AS(
 SELECT... -- начальное значение
UNION ALL -- рекурсивная часть
 SELECT...FROM имя …

)
```

Рекурсия

Рекурсивное табличное выражение имеет следующий вид:

```
WITH RECURSIVE имя(список столбцов)
AS(
 SELECT... -- начальное значение
UNION ALL -- рекурсивная часть
 SELECT...FROM имя …
WHERE условие продолжения повторения
)
```

Рекурсия

Пример. С помощью запроса получим последовательные значения от 1 до 20.

Рекурсия

Пример. С помощью запроса получим последовательные значения от 1 до 20.

```
WITH RECURSIVE create_number(number)
AS (




)
```

Рекурсия

Пример. С помощью запроса получим последовательные значения от 1 до 20.

```
WITH RECURSIVE create_number(number)
AS (
    SELECT 1 /* начальный элемент рекурсии*/




)
```

Рекурсия

Пример. С помощью запроса получим последовательные значения от 1 до 20.

```
WITH RECURSIVE create_number(number)
AS (
      SELECT 1 /* начальный элемент рекурсии*/
   UNION ALL



)
```

Рекурсия

Пример. С помощью запроса получим последовательные значения от 1 до 20.

```
WITH RECURSIVE create_number(number)
AS (
      SELECT 1 /* начальный элемент рекурсии*/
    UNION ALL
      SELECT number + 1   /* действие рекурсии*/
      FROM   create_number

)
```

Рекурсия

Пример. С помощью запроса получим последовательные значения от 1 до 20.

```
WITH RECURSIVE create_number(number)
AS (
      SELECT 1 /* начальный элемент рекурсии*/
   UNION ALL
      SELECT number + 1   /* действие рекурсии*/
      FROM   create_number
   WHERE  number < 20  /* условие продолжения рекурсии*/
)
```

Рекурсия

Пример. С помощью запроса получим последовательные значения от 1 до 20.

```
WITH RECURSIVE create_number(number)
AS (
      SELECT 1 /* начальный элемент рекурсии*/
    UNION ALL
      SELECT number + 1   /* действие рекурсии*/
      FROM   create_number
    WHERE  number < 20  /* условие продолжения рекурсии*/
)
SELECT number
FROM   create_number;
```

Рекурсия

Результат:

```
+--------+
| number |
+--------+
| 1      |
| 2      |
| 3      |
| 4      |
| 5      |
| 6      |
| 7      |
| 8      |
| 9      |
| 10     |
| 11     |
| 12     |
| 13     |
| 14     |
| 15     |
| 16     |
| 17     |
| 18     |
| 19     |
| 20     |
+--------+
```

Рекурсия

Как выполняется рекурсивное табличное выражение?

Шаг 1. В результат запроса заносится начальный элемент рекурсии:

```
SELECT 1
```

```
+--------+
| number |
+--------+
| 1      |
+--------+
```

Рекурсия

Как выполняется рекурсивное табличное выражение?

Шаг 2. К уже сформированному результату выполнения запроса присоединяется следующая строка.

```
UNION ALL
    SELECT number + 1
    FROM   create_number
```

```
+--------+
| number |
+--------+
| 1      |
| 2      |
+--------+
```

Рекурсия

Как выполняется рекурсивное табличное выражение?

Шаг 3. Проверяется условие продолжения рекурсии.

```
UNION ALL
    SELECT number + 1
    FROM   create_number
WHERE  number < 20
```

```
+--------+
| number |
+--------+
| 1      |
| 2      |
+--------+
```

Если условие Истина – продолжается выполнение рекурсивного табличного выражения.

Рекурсия

Как выполняется рекурсивное табличное выражение?

Шаг 5-6. К уже сформированному результату выполнения запроса присоединяется следующая строка и проверяется условие продолжения рекурсии.

```
UNION ALL
    SELECT number + 1
    FROM   create_number
WHERE  number < 20
```

```
+--------+
| number |
+--------+
| 1      |
| 2      |
| 3      |
+--------+
```

Если условие Истина – продолжается выполнение рекурсивного табличного выражения.

Рекурсия

Как выполняется рекурсивное табличное выражение?

Последний шаг. К уже сформированному результату выполнения запроса присоединяется следующая строка и проверяется условие продолжения рекурсии.

```
UNION ALL
    SELECT number + 1
    FROM   create_number
WHERE  number < 20
```

```
+--------+
| number |
+--------+
| 1      |
| …      |
| 20     |
+--------+
```

Если условие Ложно – выполнение рекурсивного табличного выражения заканчивается.

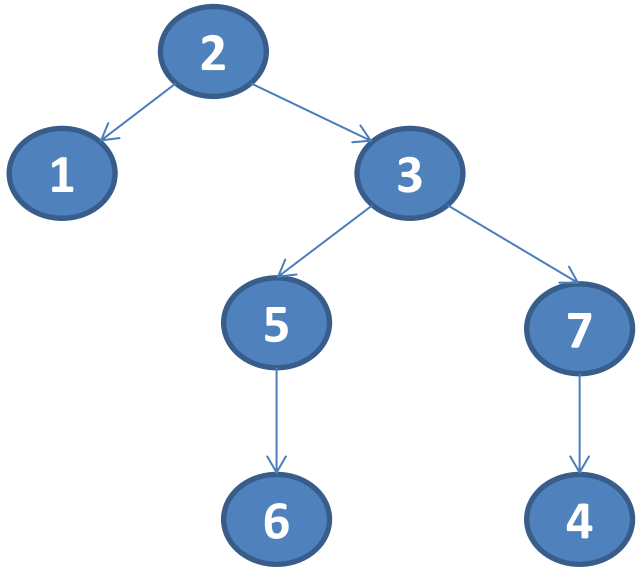
Применение рекурсивных CTE

1. Для обработки данных, представленных в иерархическом виде.

Применение рекурсивных CTE

Пример. Пусть у нас есть список сотрудников, каждый сотрудник имеет своего менеджера. При чем один и тот же сотрудник может быть подчиненным одному сотруднику и быть менеджером другого.

Эту зависимость можно представить в виде иерархической структуры:

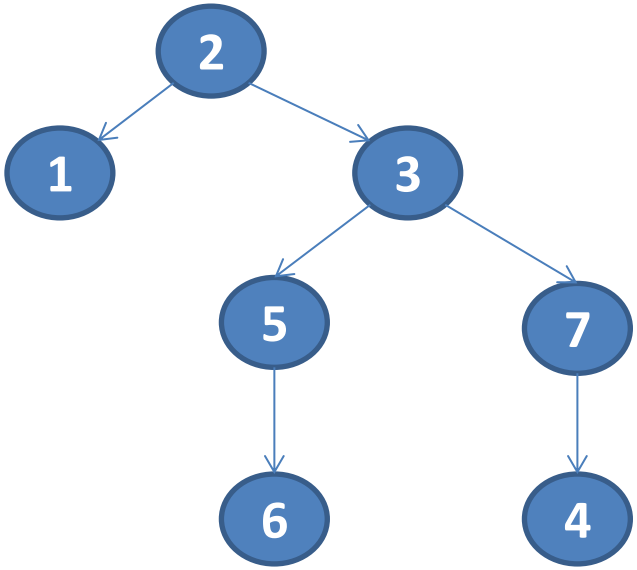


Применение рекурсивных CTE

Оформим эту зависимость в виде таблицы.

```
CREATE TABLE manager(
    staff_id INT,
    manager_id INT);

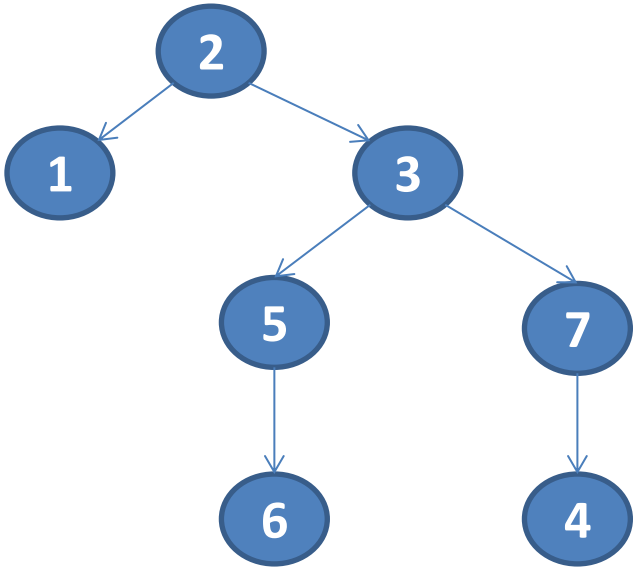
INSERT INTO manager
VALUES
(2, NULL), (1, 2), (3, 2), (5, 3), (7, 3), (6, 5), (4, 7);
```



Применение рекурсивных CTE

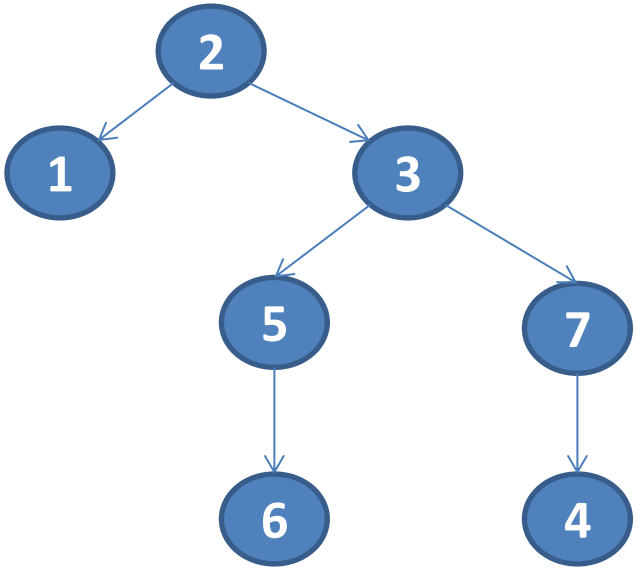
Оформим эту зависимость в виде таблицы.

staff_id	manager_id
2	NULL
1	2
3	2
5	3
7	3
6	5
4	7



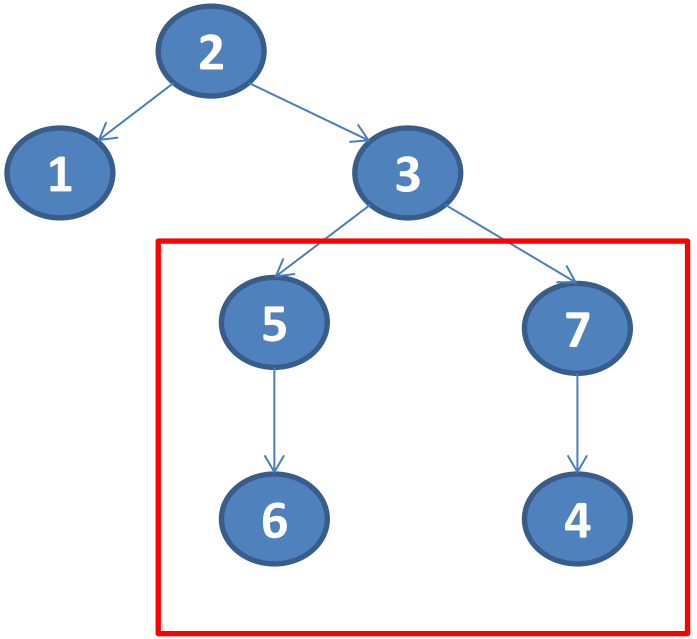
Применение рекурсивных CTE

Необходимо найти, кто подчиняется сотруднику с **id** 3.



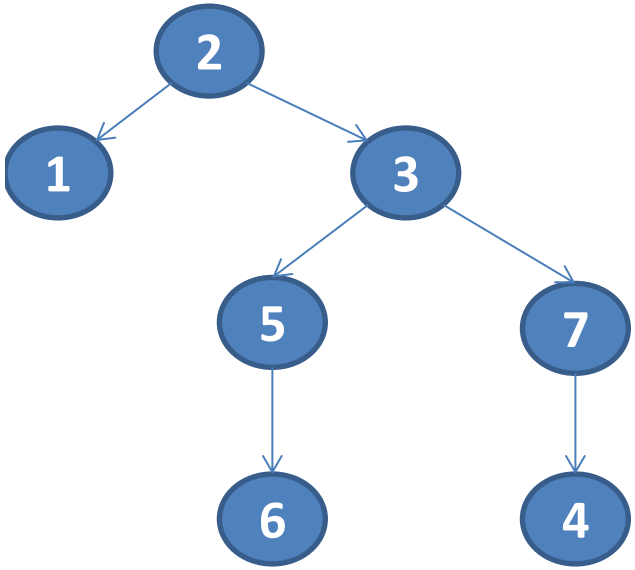
Применение рекурсивных CTE

Необходимо найти, кто подчиняется сотруднику с **id** 3



Применение рекурсивных CTE

Необходимо найти, кто подчиняется сотруднику с **id** 3.



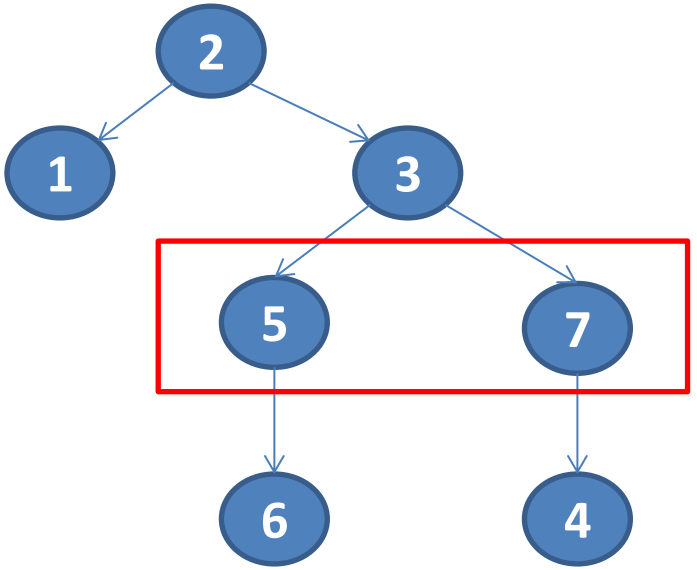

```
WITH RECURSIVE who_is_sub AS (

)
SELECT * FROM who_is_sub;
```

Применение рекурсивных CTE

Необходимо найти, кто подчиняется сотруднику с **id** 3.

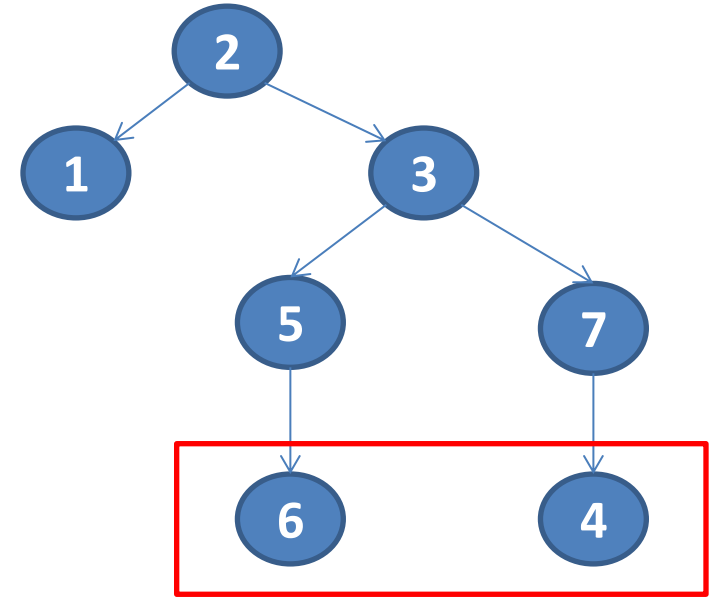
```
WITH RECURSIVE who_is_sub AS (
    SELECT staff_id, manager_id
    FROM  manager
    WHERE manager_id = 3
)
SELECT * FROM who_is_sub;
```



Применение рекурсивных CTE

Необходимо найти, кто подчиняется сотруднику с **id** 3.

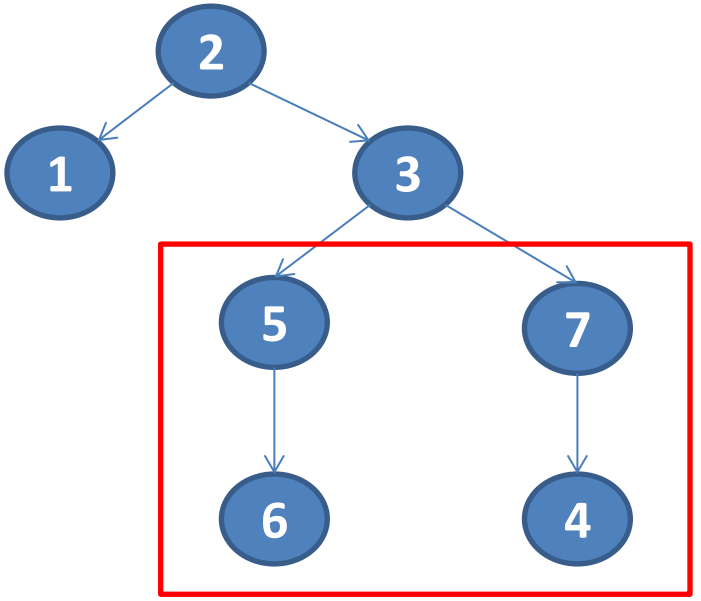
```
WITH RECURSIVE who_is_sub AS (
    SELECT staff_id, manager_id
    FROM  manager
    WHERE manager_id = 3
   UNION ALL
    SELECT manager.staff_id,
          manager.manager_id
    FROM
       manager
       INNER JOIN who_is_sub
       ON who_is_sub.staff_id =  manager.manager_id
)
SELECT * FROM who_is_sub;
```



Применение рекурсивных CTE

Необходимо найти, кто подчиняется сотруднику с **id** 3.

staff_id	manager_id
5	3
7	3
6	5
4	7



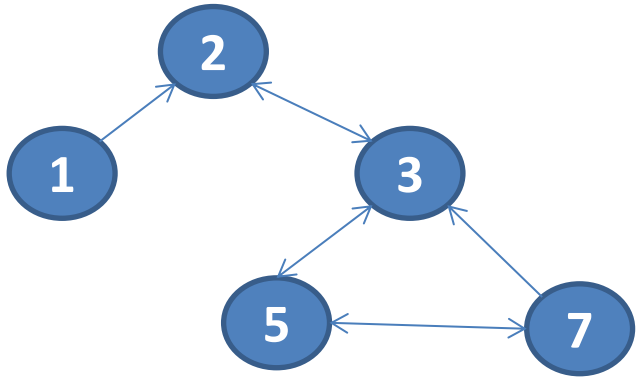
Применение рекурсивных CTE

1. Для обработки данных, представленных в иерархическом виде.
2. Для обработки данных, представленных в виде графа.

Применение рекурсивных CTE

Пример. Пусть у нас есть список городов. Город связан дорогами с другими городами. Дороги могут быть с односторонним и двухсторонним движением.

Эту зависимость можно представить в виде графа:

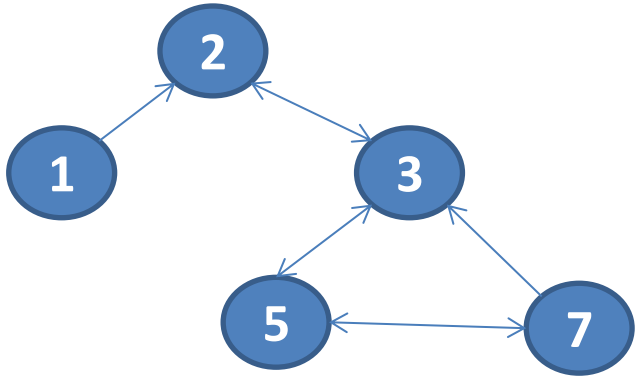


Применение рекурсивных CTE

Оформим эту зависимость в виде таблицы.

```
CREATE TABLE bus_routes(
    sity_beg_id INT,
    sity_end_id INT);

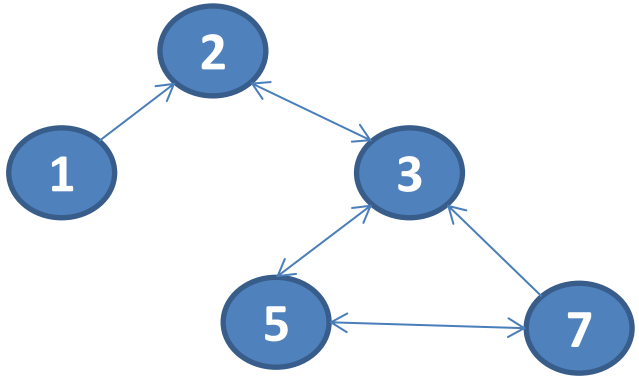
INSERT INTO bus_routes
VALUES
(1, 2), (2, 3), (3, 2), (3, 5), (5, 3), (5, 7), (7, 5), (7, 3);
```



Применение рекурсивных CTE

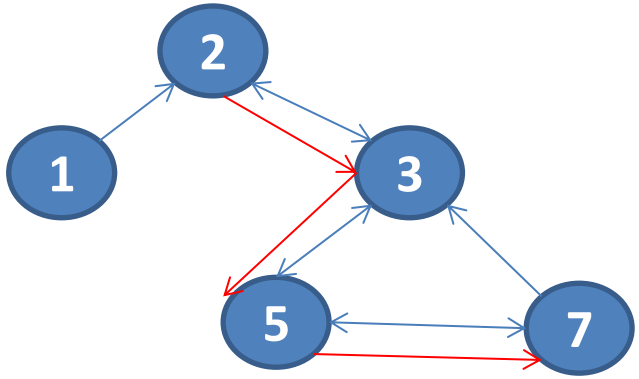
Оформим эту зависимость в виде таблицы.

sity_beg_id	sity_end_id
1	2
2	3
3	2
3	5
5	3
5	7
7	5
7	3



Применение рекурсивных CTE

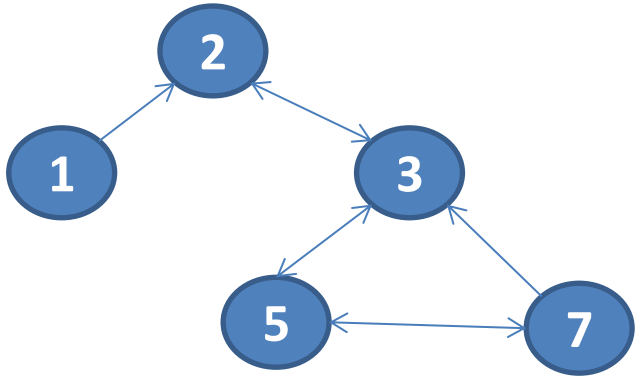
Необходимо найти, в какие города можно добраться из города с id 2.



Применение рекурсивных CTE

Необходимо найти, в какие города можно добраться из города с id 2.

```
WITH RECURSIVE paths (cur_dest)
 AS (
```

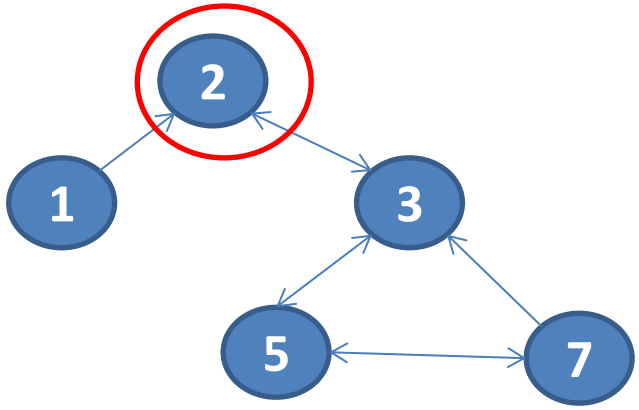



```
)
SELECT * FROM paths;
```

Применение рекурсивных CTE

Необходимо найти, в какие города можно добраться из города с id 2.

```
WITH RECURSIVE paths (cur_dest)
 AS (
  SELECT sity_beg_id
    FROM bus_routes
    WHERE sity_beg_id = 2
```

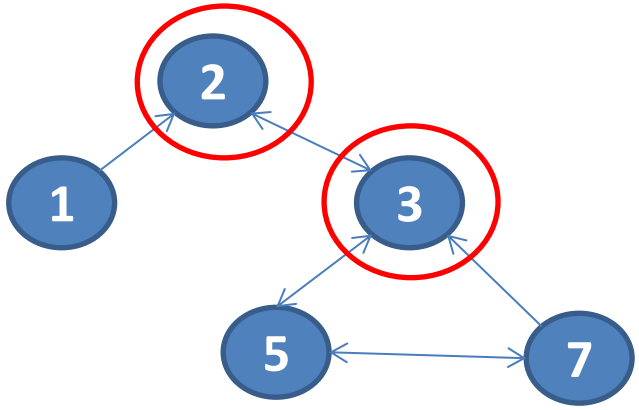



```
)
SELECT * FROM paths;
```

Применение рекурсивных CTE

Необходимо найти, в какие города можно добраться из города с id 2.

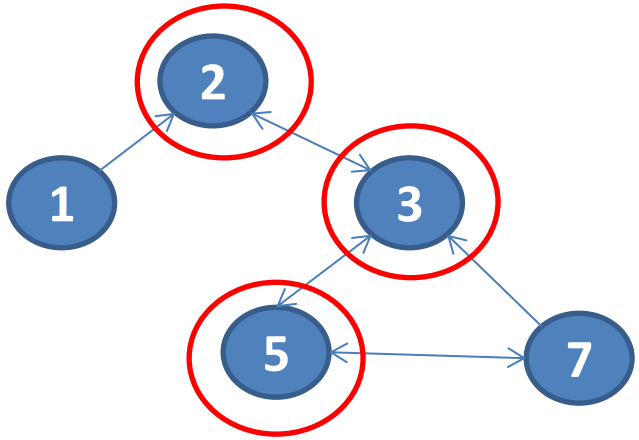
```
WITH RECURSIVE paths (cur_dest)
 AS (
  SELECT sity_beg_id
    FROM bus_routes
    WHERE sity_beg_id = 2
  UNION
  SELECT bus_routes.sity_end_id
   FROM
      paths JOIN bus_routes
      ON paths.cur_dest = bus_routes.sity_beg_id
)
SELECT * FROM paths;
```



Применение рекурсивных CTE

Необходимо найти, в какие города можно добраться из города с id 2.

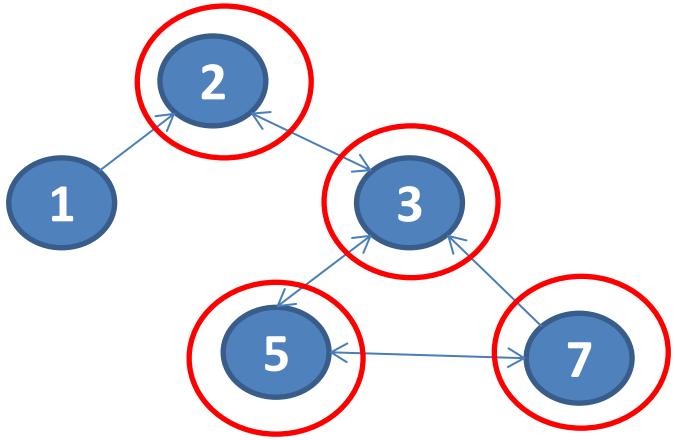
```
WITH RECURSIVE paths (cur_dest)
 AS (
  SELECT sity_beg_id
    FROM bus_routes
    WHERE sity_beg_id = 2
  UNION
  SELECT bus_routes.sity_end_id
   FROM
      paths JOIN bus_routes
      ON paths.cur_dest = bus_routes.sity_beg_id
)
SELECT * FROM paths;
```



Применение рекурсивных CTE

Необходимо найти, в какие города можно добраться из города с id 2.

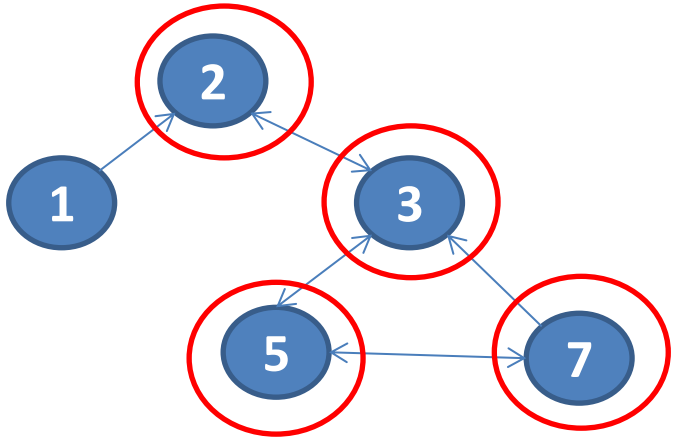
```
WITH RECURSIVE paths (cur_dest)
 AS (
  SELECT sity_beg_id
    FROM bus_routes
    WHERE sity_beg_id = 2
  UNION
  SELECT bus_routes.sity_end_id
   FROM
      paths JOIN bus_routes
      ON paths.cur_dest = bus_routes.sity_beg_id
)
SELECT * FROM paths;
```



Применение рекурсивных CTE

Необходимо найти, в какие города можно добраться из города с id 2.

```
+----------+
| cur_dest |
+----------+
| 2        |
| 3        |
| 5        |
| 7        |
+----------+
```



Применение рекурсивных CTE

Необходимо найти, в какие города можно добраться из города с id 2.
а также указать, через какие города проходит путь.

```
WITH RECURSIVE paths (cur_path, cur_dest)
 AS (




)
SELECT * FROM paths;
```

Применение рекурсивных CTE

Необходимо найти, в какие города можно добраться из города с id 2.
а также указать, через какие города проходит путь.

```
WITH RECURSIVE paths (cur_path, cur_dest)
 AS (
  SELECT  sity_beg_id , sity_beg_id
    FROM bus_routes
    WHERE sity_beg_id = 2




)
SELECT * FROM paths;
```

Применение рекурсивных CTE

Необходимо найти, в какие города можно добраться из города с id 2.
а также указать, через какие города проходит путь.

```
WITH RECURSIVE paths (cur_path, cur_dest)
 AS (
  SELECT CAST(sity_beg_id AS CHAR(100)), sity_beg_id
    FROM bus_routes
    WHERE sity_beg_id = 2




)
SELECT * FROM paths;
```

Применение рекурсивных CTE

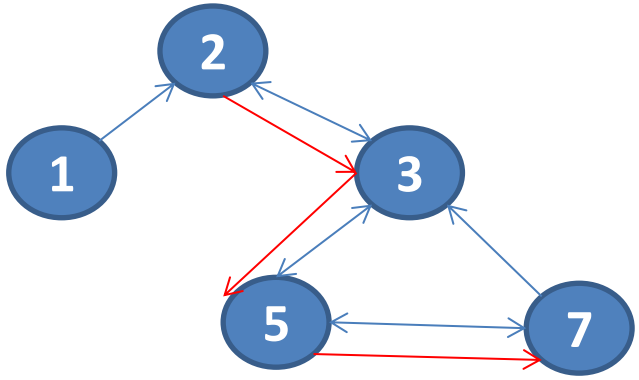
Необходимо найти, в какие города можно добраться из города с id 2.
а также указать, через какие города проходит путь.

```
WITH RECURSIVE paths (cur_path, cur_dest)
 AS (
  SELECT CAST(sity_beg_id AS CHAR(100)), sity_beg_id
    FROM bus_routes
    WHERE sity_beg_id = 2
  UNION
  SELECT CONCAT(cur_path, ', ', sity_end_id), sity_end_id
   FROM
      paths JOIN bus_routes
      ON cur_dest = sity_beg_id  AND LOCATE(sity_end_id, cur_path) = 0
)
SELECT * FROM paths;
```

Применение рекурсивных CTE

Необходимо найти, в какие города можно добраться из города с id 2.
а также указать, через какие города проходит путь.

cur_path	cur_dest
2	2
2, 3	3
2, 3, 5	5
2, 3, 5, 7	7



Применение рекурсивных CTE

1. Для обработки данных, представленных в иерархическом виде.
2. Для обработки данных, представленных в виде графа.
3. Для создания «накапливаемых» таблиц совместно с CROSS JOIN

Применение рекурсивных CTE

Пример. Пусть у нас есть расписание работы врача. Необходимо создать таблицу для записи пациента на прием, если на каждого человека отводится по пол часа, а все врачи работают с 8 до 15 часов.

Применение рекурсивных CTE

Пример. Пусть у нас есть расписание работы врача. Необходимо создать таблицу для записи пациента на прием, если на каждого человека отводится по пол часа, а все врачи работают с 8 до 15 часов.

```
CREATE TABLE schedule_doctor(
   doctor_id INT,
    schedule_date DATE
);

INSERT INTO schedule_doctor(doctor_id, schedule_date)
VALUES
(1,'2023-07-24'), (2,'2023-07-24'), (3,'2023-07-25'), (1,'2023-07-25');
```

Применение рекурсивных CTE

Пример. Пусть у нас есть расписание работы врача. Необходимо создать таблицу для записи пациента на прием, если на каждого человека отводится по пол часа, а все врачи работают с 8 до 15 часов.

doctor_id	schedule_date
1	2023-07-24
2	2023-07-24
3	2023-07-25
1	2023-07-25

Применение рекурсивных CTE

Шаг 1. С помощью рекурсивного табличного выражения создадим временные интервалы: 8:00, 8:30 и т.д.

Применение рекурсивных CTE

Шаг 1. С помощью рекурсивного табличного выражения создадим временные интервалы: 8:00, 8:30 и т.д.

```
WITH RECURSIVE create_time(time_s)
AS (




)
SELECT * from create_time;
```

Применение рекурсивных CTE

Шаг 1. С помощью рекурсивного табличного выражения создадим временные интервалы: 8:00, 8:30 и т.д.

```
+----------+
| time_s   |
+----------+
| 8:00:00  |
+----------+
```

```
WITH RECURSIVE create_time(time_s)
AS (
   SELECT  '8:00:00'




)
SELECT * from create_time;
```

Применение рекурсивных CTE

Шаг 1. С помощью рекурсивного табличного выражения создадим временные интервалы: 8:00, 8:30 и т.д.

```
WITH RECURSIVE create_time(time_s)
AS (
   SELECT  CAST('8:00:00' AS TIME)



)
SELECT * from create_time;
```

```
+----------+
| time_s   |
+----------+
| 8:00:00  |
+----------+
```

Применение рекурсивных CTE

Шаг 1. С помощью рекурсивного табличного выражения создадим временные интервалы: 8:00, 8:30 и т.д.

```
WITH RECURSIVE create_time(time_s)
AS (
   SELECT  CAST('8:00:00' AS TIME)
   UNION ALL
   SELECT ADDTIME(time_s,'0:30:00')
   FROM   create_time

)
SELECT * from create_time;
```

```
+----------+
| time_s   |
+----------+
| 8:00:00  |
| 8:30:00  |
+----------+
```

Применение рекурсивных CTE

Шаг 1. С помощью рекурсивного табличного выражения создадим временные интервалы: 8:00, 8:30 и т.д.

```
WITH RECURSIVE create_time(time_s)
AS (
   SELECT  CAST('8:00:00' AS TIME)
   UNION ALL
   SELECT ADDTIME(time_s,'0:30:00')
   FROM   create_time
   WHERE   time_s  < CAST('15:00:00' AS TIME)
)
SELECT * from create_time;
```

```
+----------+
| time_s   |
+----------+
| 8:00:00  |
| 8:30:00  |
+----------+
```

Применение рекурсивных CTE

Шаг 1. С помощью рекурсивного табличного выражения создадим временные интервалы: 8:00, 8:30 и т.д.

```
WITH RECURSIVE create_time(time_s)
AS (
   SELECT  CAST('8:00:00' AS TIME)
   UNION ALL
   SELECT ADDTIME(time_s,'0:30:00')
   FROM   create_time
   WHERE   time_s  < CAST('15:00:00' AS TIME)
)
SELECT * from create_time;
```

```
+----------+
| time_s   |
+----------+
| 8:00:00  |
| 8:30:00  |
| 9:00:00  |
+----------+
```

Применение рекурсивных CTE

Шаг 1. С помощью рекурсивного табличного выражения создадим временные интервалы: 8:00, 8:30 и т.д.

```
WITH RECURSIVE create_time(time_s)
AS (
   SELECT  CAST('8:00:00' AS TIME)
   UNION ALL
   SELECT ADDTIME(time_s,'0:30:00')
   FROM   create_time
   WHERE   time_s  < CAST('15:00:00' AS TIME)
)
SELECT * from create_time;
```

```
+----------+
| time_s   |
+----------+
| 8:00:00  |
| 8:30:00  |
| 9:00:00  |
| 9:30:00  |
| 10:00:00 |
| 10:30:00 |
| 11:00:00 |
| 11:30:00 |
| 12:00:00 |
| 12:30:00 |
| 13:00:00 |
| 13:30:00 |
| 14:00:00 |
| 14:30:00 |
| 15:00:00 |
```

Применение рекурсивных CTE

Шаг 2. Объединяем табличное выражение с таблицей расписание.

```
SET @end_time = SUBTIME('15:00:00', '0:30:00')
WITH RECURSIVE create_time(time_s)
AS (
   SELECT  CAST('8:00:00' AS TIME)
   UNION ALL
   SELECT ADDTIME(time_s,'0:30:00')
   FROM   create_time
   WHERE   time_s  < @end_time
)
SELECT doctor_id, schedule_date, time_s
FROM schedule_doctor, create_time
ORDER BY 1, 2, 3
```

Применение рекурсивных CTE

Результат (60 записей):

doctor_id	schedule_date	time_s
1	2023-07-24	8:00:00
1	2023-07-24	8:30:00
1	2023-07-24	9:00:00
	…	
1	2023-07-24	15:00:00
1	2023-07-25	8:00:00
	…	
1	2023-07-25	15:00:00
2	2023-07-24	8:00:00
2	2023-07-24	8:30:00
	…	

Спасибо за внимание!